

75. Une fonction qui rend une valeur

Ce qui entre, ce qui sort — et ce qu'on ne réécrit pas deux fois.

1. Ce qu'il faut comprendre

Les deux appels rendent 70. Sans la fonction, il faudrait écrire le `if` deux fois — et le jour où l'on veut borner l'écart à 60, le corriger aux deux endroits.

Le résultat s'utilise dans un calcul : `nombre(10, 6, distance(a, b) * 2)` marche, comme n'importe quelle expression.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **570 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
distance()     23 o ← nombre() - ta fonction
main()         56 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         240 o ← CopierTuiles - 15 lettres sur 43
"ECART"        5 o ← texte() - un texte gravé
```

Lettres gravées : ACERT0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre()  x 2   16 o sur place, dans main() + distance() (23 o), EcrireNombre (33 o)
texte()   x 1   11 o sur place, dans main() + EcrireTexte (10 o)
image()   x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
distance() x 2   22 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : calcule l'écart entre deux nombres avec une fonction qui RÉPOND.  
//  
// uint8_t distance(...) : le uint8_t devant le nom dit ce que la fonction rend.  
// return X; = « la réponse est X » (et la fonction s'arrête là).  
// Pourquoi deux cas ? Un uint8_t n'a pas de nombres négatifs :  
// 30 - 100 donnerait 186, pas -70. On soustrait donc toujours le petit du grand.  
  
#include <texte>    // écrit un texte à l'écran  
#include <nombre>   // écrit un nombre en chiffres
```

2. UNE FONCTION — DISTANCE()

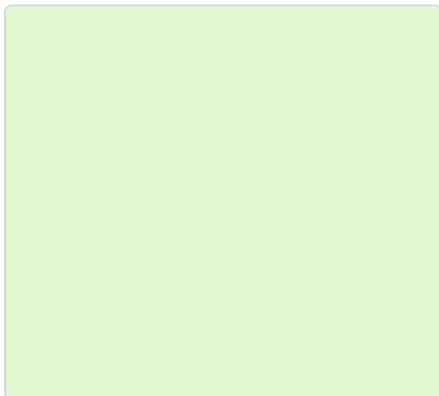
```
uint8_t distance(uint8_t a, uint8_t b) {  
    if (a > b) return a - b;    // a est le plus grand : a - b.  
    return b - a;              // sinon : b - a.  
}  
// distance(30, 100) = 70 ; distance(100, 30) = 70.
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

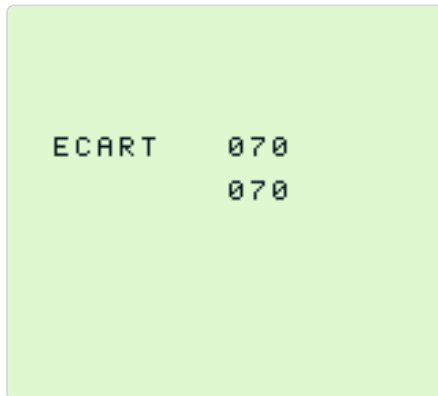
```
int main() {                      // Le jeu commence ici.  
    texte(2, 6, "ECART");  
    nombre(10, 6, distance(30, 100)); // La réponse s'affiche : 070.  
    nombre(10, 8, distance(100, 30)); // Dans l'autre sens : 070 aussi.  
  
    while (true) {                // Répète pour toujours :  
        image();                  // attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

070 deux fois — l'ordre des arguments ne change rien.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le premier sens donne 70

✓ le second aussi

✓ l'étiquette est là

Ensuite : **76. Un argument par défaut** — chapitre 11 : Découper son programme. · [gameboy3](#) — écrire une cartouche Game Boy en C++